

Reference repositories — round 2

Purpose. Round 1 covered the engineering stack. Round 2 covers the two questions that decide whether a subscription app makes or loses money, plus the offline-write problem the app has not hit yet and will.

Method. GitHub API metadata + README/source review. Stars as of 2026-09-15.

Why these and not more. Each entry answers a question this build has already deferred. A reference that does not change a decision was not worth the read — the same rule as round 1.

1. `RevenueCat/purchases-ios` — 3,069★ · MIT

What it is. The incumbent in-app purchase abstraction, in Swift.

Lesson applied (the one that matters most). RevenueCat’s entire product exists because the StoreKit failure mode is not “a purchase fails”. It is: **a purchase succeeds and the server never hears about it**. The client has a receipt, the customer has been charged, and the app’s own database says they are on the free tier. That is the single most expensive bug class in subscription software, and this app has the exact shape that produces it:

```
StoreKit charges the card
→ app POSTs the receipt to /api/mobile/v1/... ← THIS CAN FAIL
→ server records the entitlement
→ app refreshes and shows Pro
```

Between step 1 and step 3 there is a window, and **nothing in the current build closes it**. If step 2 times out, the user has paid and sees the free tier. `finishTransaction` is called on `onPurchaseSuccess`, which is worse: finishing the transaction tells StoreKit the app has done its job, so StoreKit will never re-deliver it.

This is now the highest-priority gap in the mobile app, above anything cosmetic, and it is recorded as such in the audit’s remaining-gaps list.

The fix, in shape (not yet implemented): 1. Do NOT `finishTransaction` until the SERVER has acknowledged. StoreKit will re-deliver an unfinished transaction on next launch, which makes the client’s retry unnecessary. 2. `appAccountToken` — set it to the app’s user id at purchase time. StoreKit carries it on the transaction, so even a purchase made while signed out can be reconciled to an account later. `expo-iap`’s `PurchaseIOS` already exposes `appAccountToken`. 3. A server-side reconciliation sweep: on every app launch and on a cron, `getAvailablePurchases()` and POST anything the server does not know about. Idempotent, so it can run often.

Lesson NOT applied. RevenueCat itself. It would be a third party holding the subscription record, a revenue share, and a dependency in the entitlement path. The reconciliation logic is ~150 lines against an API this app already wraps.

2. `russell-archer/StoreHelper` — 466★ · MIT

What it is. A worked StoreKit 2 implementation used as teaching material.

Lesson applied. Its treatment of subscription state is a state machine, not a boolean: `active`, `expired`, `inGracePeriod`, `inBillingRetry`, `revoked`, `upgraded`. The app currently models a tier as a single value, which cannot express “paid, but the card failed and Apple is retrying” — and that state is the one where a customer is most likely to contact support and most likely to churn.

Lesson applied (diagnosis). Its `Transaction.updates` listener is the answer to a question this app has not asked: what happens when a purchase or renewal happens **while the app is not running**? For a subscriptions product the honest answer is that the entitlement must be re-derived on every foreground from `Transaction.currentEntitlements`, not cached from the last purchase event.

Gap this exposes. There is no `Transaction.updates` listener in the app. Recorded.

3. `tikhop/TPInAppReceipt` — 718★ · MIT

What it is. Local receipt reading and validation for StoreKit 1/2.

Lesson applied (negative, and it is a security one). The temptation is to validate the receipt ON the device and flip a local “isPro” flag — the SDK makes it easy and it removes the server round-trip. That is a client-side paywall, which this repo forbids in CLAUDE.md rule 3, and it is worse on a native client than on the web because the binary is on the user’s device and can be patched.

So: the receipt is an **input to the server**, never an authority on the client. The app’s own `lib/entitlements.ts` header already says the client is presentational only; this is where that rule becomes expensive, and it must hold.

Lesson applied (scope). Its local-validation path IS useful for one thing: deciding whether to show a “restore purchases” affordance. A locally readable receipt means the button can be offered without a network call, and a failed restore can be distinguished from “nothing to restore”.

4. `Expensify/react-native-onyx` — 231★ · MIT

What it is. Expensify’s persistent, offline-first state layer for React Native.

Lesson applied. Onyx’s core discipline is that every write goes through ONE queue with a persisted intent, and the UI reads only from the local store — never from the network directly. The app follows the read half of that (the cache is the read path; `fromCache` is labelled). It does not yet follow the write half, because the only writes today are watchlist follow/unfollow.

The gap, stated plainly. `watchlistFollow` is a fire-and-forget POST. Offline, it fails and the user sees nothing change. Worse, if it succeeds but the response is lost, the user taps again and gets a duplicate. The app has no queue and no conflict rule.

Lesson applied (the shape of the fix). A write queue needs three things and the app has none: an intent persisted BEFORE the request, an idempotency key the server understands, and a defined resolution when the local and server states disagree. Without the third, a queue turns one bug into two.

5. `redux-offline/redux-offline` — 6,128★ · MIT

What it is. The long-standing offline-first pattern for React Native.

Lesson applied, and then rejected for this app. Its model is an optimistic action queue with automatic replay and rollback. That is right for a product where the user’s intent is authoritative (a todo list: if I typed it, it is true).

It is WRONG here, for the same reason the client may only remove rows and never add them: **the server is the authority on every fact this product publishes**. An optimistically-followed pick that the server then refuses would put a row on screen that the engine declined to publish. That is the `adverse-edge-suppression` problem in a different costume.

The adopted rule: exactly one write in this app may be optimistic, and it is the watchlist, because a watchlist entry is a fact about the USER, not about the model. Every other mutation waits for the server. That line is worth writing down before the queue is built, because it is the decision that is hard to reverse.

6. `berty/berty` — 9,299★ · Apache-2.0

What it is. A peer-to-peer messaging app built to work with or without a network. Included not for its domain but for its conflict model.

Lesson applied. Berty’s documentation is unusually honest that offline-first is a *distributed systems* problem wearing a UI costume, and that most apps adopting the pattern do not have the problem they think they have. The question to ask first is not “how do we sync” but “what is the authoritative clock, and what happens when two clocks disagree”.

Applied to this app: the authoritative clock is the SERVER’s `lastRefresh`, and there is exactly one of it. A device clock is used only to render “12 min ago”, which is why `relativeTime` clamps a future timestamp to “just now” rather than rendering a negative age. That clamp is not cosmetic; it is the app refusing to let two clocks disagree on screen.

Lesson NOT applied: CRDTs, vector clocks, or any multi-master merge. There is one writer per fact. Adopting a merge algorithm would be solving a problem the architecture already prevents.

What round 2 changed, in one list

Finding	Consequence
A purchase can succeed with the server never hearing about it, and the current code finishes the transaction anyway	Highest-priority gap in the mobile app. Do not finish until the server acknowledges; set <code>appAccountToken</code> ; add a reconciliation sweep.
Subscription state is a state machine, not a boolean	Grace period and billing-retry are not representable today.
There is no <code>Transaction.updates</code> listener	An entitlement change while the app is closed is invisible until the next server fetch.
Local receipt validation would be a rule-3 violation	The receipt is server input only. Non-negotiable.
The watchlist write is fire-and-forget with no queue	Offline it silently does nothing; on a lost response it duplicates.
The watchlist is the ONLY legitimate optimistic write	Every other fact belongs to the server. Write this down before building the queue.
One authoritative clock, and <code>relativeTime</code> already clamps to it	Do not add a merge algorithm.

Round 3 (queued)

1. **App Store review outcomes for gambling-adjacent apps** — what actually gets rejected, from developers' own post-mortems rather than from the published guidelines.
2. **Accessibility-first charting** — a reliability diagram for VoiceOver that does not read sixteen numbers.
3. **App Intents and Siri** — whether “how many picks are open” is a better surface than a widget.
4. **Privacy-preserving analytics** — whether the product-interaction collection earns its place at all, given the app collects it unlinked.
5. **E2E on a simulator** — Maestro or Detox, which needs the Mac that this build still needs.